CS683 Programming Assignment -2 report

Submitted by: 24B1015, 24B0995, 24B1021, 24B1037

1 Task 1: Stride Pattern Prefetching

1.1 Task 1A: Understanding the IP-stride prefetcher

1.1.1 How the prefetcher works

The IP-stride prefetcher (`prefetcher/ip_stride.l1d_pref`) sits at the L1 data cache. Its idea is that a single load or store instruction, identified by its instruction pointer (IP), tends to walk through memory with a constant stride (for example, a loop over an array). It remembers the last address each IP touched, computes the stride between consecutive accesses of that IP, and once the same stride is seen twice in a row it prefetches the next few addresses along that stride. The code is walked through below, section by section; the line numbers in the listings are the line numbers in the source file.

1. Parameters and the tracker entry (lines 4–30).

```
4  #define IP_TRACKER_COUNT 24
5  #define PREFETCH_DEGREE 3
6  #define MASK_IP 0xFF
7
8  class IP_TRACKER {
9    public:
10     // the IP we're tracking
11     uint64_t ip;
12
13     // the last address accessed by this IP
14     uint64_t last_cl_addr;
15
16     // the stride between the last two addresses accessed by this IP
17     int64_t last_stride;
18
19     // use LRU to evict old IP trackers
20     uint32_t lru;
21
22     IP_TRACKER () {
23         ip = 0;
24         last_cl_addr = 0;
25         last_stride = 0;
26         lru = 0;
27     };
28 };
29
30 IP_TRACKER trackers[IP_TRACKER_COUNT];
```

- `IP_TRACKER_COUNT = 24`: the prefetching table has only 24 entries.
- `PREFETCH_DEGREE = 3`: when the prefetcher fires it issues 3 prefetches. The degree is fixed and never changes.
- `MASK_IP = 0xFF`: only the low 8 bits of the IP are used to identify an instruction. Different instructions whose IPs share the low 8 bits therefore share one tracker (aliasing).
- Each `IP_TRACKER` entry stores: the (masked) IP it belongs to; the cache-line address of the last access by that IP (`last_cl_addr`); the stride observed between its last two accesses (`last_stride`, signed, in cache lines); and an LRU rank used to choose which entry to replace when the table is full.

- The table `trackers[]` is a single global array, so it is shared by everything that uses this prefetcher (fine for the single-core configuration used here).

2. Initialisation (lines 33–39).

```
33 void CACHE::l1d_prefetcher_initialize()
34 {
35     cout << "CPU " << cpu << " L1D IP-based stride prefetcher" << endl;
36     for (int i=0; i<IP_TRACKER_COUNT; i++)
37             trackers[i].lru = i;
38 
39 }
```

This function is called once when the simulation starts. It prints a banner and gives the 24 entries the distinct LRU ranks $0, 1, \ldots, 23$. Rank 0 means most recently used and rank 23 (`IP_TRACKER_COUNT-1`) means least recently used, so at any time the ranks form a permutation of $0 \ldots 23$ and exactly one entry is the replacement victim. All entries start with `ip = 0`, `last_cl_addr = 0` and `last_stride = 0`.

3. Entry point: cache-line address and IP (lines 41–46).

```
41 void CACHE::l1d_prefetcher_operate(uint64_t addr, uint64_t ip, uint8_t cache_hit, uint8_t
       type, uint8_t critical_ip_flag)
42 {
43     // check for a tracker hit
44     uint64_t cl_addr = addr >> LOG2_BLOCK_SIZE;
45 
46     ip = ip & MASK_IP;
```

`l1d_prefetcher_operate` is called by the L1D for each access it handles. The prefetcher works on cache lines, not bytes: shifting the byte address right by `LOG2_BLOCK_SIZE` drops the offset inside the 64-byte line, giving `cl_addr`. The IP is reduced to its low 8 bits. The arguments `cache_hit` and `type` are not used in this file, so the prefetcher trains on every access it is called for, whether it hit or missed in the L1D.

4. Looking up the tracker (lines 48–52).

```
48     int index = -1;
49     for (index=0; index<IP_TRACKER_COUNT; index++) {
50         if (trackers[index].ip == ip)
51             break;
52     }
```

The table is fully associative: a linear search compares the masked IP against every entry. If an entry matches, the loop `break`s with `index` pointing at it. If none matches, the loop runs to the end and `index` equals `IP_TRACKER_COUNT`.

5. A new IP: allocating a tracker (lines 54–74).

```
54      // this is a new IP that doesn't have a tracker yet, so allocate one
55      if (index == IP_TRACKER_COUNT) {
56
57          for (index=0; index<IP_TRACKER_COUNT; index++) {
58              if (trackers[index].lru == (IP_TRACKER_COUNT-1))
59                  break;
60          }
61
62          trackers[index].ip = ip;
63          trackers[index].last_cl_addr = cl_addr;
64          trackers[index].last_stride = 0;
65
66          //cout << "[IP_STRIDE] MISS index: " << index << " lru: " << trackers[index].lru
        << " ip: " << hex << ip << " cl_addr: " << cl_addr << dec << endl;
67      for (int i=0; i<IP_TRACKER_COUNT; i++) {
68              if (trackers[i].lru < trackers[index].lru)
69                  trackers[i].lru++;
70          }
71          trackers[index].lru = 0;
72
73          return; //metadata_in;
74      }
```

- **Choosing the victim (57–60).** The entry whose LRU rank is 23 (the least recently used) is selected for replacement.
- **Initialising it (62–64).** The entry is overwritten with the new IP and the current line address. `last_stride` is set to 0 because no stride is known yet.
- **LRU update (67–71).** Every entry that was more recently used than the victim (rank smaller than the victim's) moves one step towards "old" (rank +1), and the new entry becomes rank 0. Since the victim had rank 23, all other entries are incremented and the ranks remain a permutation.
- **No prefetch (73).** With only one address seen, there is no stride to compute, so the function returns without prefetching.

6. Sanity check and stride calculation (lines 76–90).

```
76      // sanity check
77      // at this point we should know a matching tracker index
78      if (index == -1)
79          assert(0);
80
81      // calculate the stride between the current address and the last address
82      // this bit appears overly complicated because we're calculating
83      // differences between unsigned address variables
84      int64_t stride = 0;
85      if (cl_addr > trackers[index].last_cl_addr)
86          stride = cl_addr - trackers[index].last_cl_addr;
87      else {
88          stride = trackers[index].last_cl_addr - cl_addr;
89          stride *= -1;
90      }
```

When execution reaches here, a tracker for this IP exists. The `assert` guards against an impossible state. The stride is the difference between the current and the previous cache-line

address of this IP, in units of cache lines. Because the addresses are unsigned, the code subtracts the smaller from the larger and negates the result if the access moved backwards, so a descending access pattern gives a negative stride.

7. Ignoring repeated lines (lines 94–96).

```
94      // don't do anything if we somehow saw the same address twice in a row
95      if (stride == 0)
96          return; //metadata_in;
```

A stride of 0 means the same cache line was accessed again (for example several accesses to the same line). There is nothing to learn or prefetch from that, so the function returns early. Note that this early return does *not* update `last_cl_addr`, `last_stride` or the LRU ranks.

8. The trigger and the prefetch loop (lines 98–117).

```
98      // only do any prefetching if there's a pattern of seeing the same
99      // stride more than once
100     if (stride == trackers[index].last_stride) {
101
102         // do some prefetching
103         for (int i=0; i<PREFETCH_DEGREE; i++) {
104             uint64_t pf_address = (cl_addr + (stride*(i+1))) << LOG2_BLOCK_SIZE;
105
106             // only issue a prefetch if the prefetch address is in the same 4 KB page
107             // as the current demand access address
108             if ((pf_address >> LOG2_PAGE_SIZE) != (addr >> LOG2_PAGE_SIZE))
109                 break;
110
111             // check the MSHR occupancy to decide if we're going to prefetch to the L1 or
     L2
112             if (MSHR.occupancy < (MSHR.SIZE>>1))
113               prefetch_line(ip, addr, pf_address, FILL_L1, 0);
114             else
115               prefetch_line(ip, addr, pf_address, FILL_L2, 0);
116         }
117     }
```

- **Trigger (100).** Prefetching only happens if the new stride equals the stored `last_stride`, i.e. the same stride has been seen twice in a row. This is the prefetcher's only form of confidence: one repetition. Since a new entry starts with `last_stride = 0`, an IP needs three accesses before its first prefetch (the first allocates the entry, the second records the stride, the third confirms it).
- **Addresses (103–104).** For $i = 0, 1, 2$ it prefetches the line at `cl_addr + stride` $\cdot(i+1)$, converted back to a byte address, so with stride s it prefetches the next three lines along the pattern.
- **Page boundary (108–109).** The prefetch address must be in the same 4 KB page as the demand access (the usual reason is that consecutive pages are not necessarily contiguous or even mapped, so a stride run cannot safely be followed across a page. Also the prefetcher does not know whether the next page is actually valid/mapped.). On the first violation the loop stops, since the later prefetches in the same direction would also be outside the page.
- **Fill level (112–115).** `prefetch_line` hands the request to the cache. If fewer than half of the L1D MSHR entries are in use the line is brought into the L1D (`FILL_L1`); otherwise it is only brought into the L2 (`FILL_L2`), so that prefetches do not use up the miss-handling resources that demand misses need.

9. Updating the entry (lines 119–128).

```
119     trackers[index].last_cl_addr = cl_addr;
120     trackers[index].last_stride = stride;
121
122     for (int i=0; i<IP_TRACKER_COUNT; i++) {
123         if (trackers[i].lru < trackers[index].lru)
124             trackers[i].lru++;
125     }
126     trackers[index].lru = 0;
127     return; //metadata_in;
128 }
```

Whether or not a prefetch was issued, the entry now remembers the current line address and the current stride, ready for the next access of this IP. The entry then becomes the most recently used: every entry that was more recent than it is aged by one and it gets rank 0. Note that only the *last* stride is kept as history; one irregular access overwrites it.

10. Fill hook and final statistics (lines 130–139).

```
130 void CACHE::l1d_prefetcher_cache_fill(uint64_t v_addr, uint64_t addr, uint32_t set,
        uint32_t way, uint8_t prefetch, uint64_t v_evicted_addr, uint64_t evicted_addr,
        uint32_t metadata_in)
131 {
132     return; //metadata_in;
133 }
134
135 void CACHE::l1d_prefetcher_final_stats()
136 {
137     cout << "CPU " << cpu << " L1D PC-based stride prefetcher final stats" << endl;
138     cout << "Degree: " << PREFETCH_DEGREE << endl;
139 }
```

`l1d_prefetcher_cache_fill` is called whenever a line is filled into the L1D. This prefetcher does not use it (the body just returns). `l1d_prefetcher_final_stats` prints the degree at the end of the simulation. There is no accuracy tracking in this prefetcher; the accuracy reported in the results comes from ChampSim's own prefetch counters.

1.1.2 Limitations

The IP-stride prefetcher only works for accesses with one constant stride per IP. Its main limitations, which motivate SPP, are:

- **No tolerance to irregularity.** Each entry stores a single stride and a single previous address, and a prefetch is issued only when $s_n = s_{n-1}$, where $s_n = A_n - A_{n-1}$. One irregular access overwrites `last_stride`, so the next access cannot match it and the stride has to be re-learned and re-confirmed. A single irregular access therefore suppresses prefetching for about two subsequent accesses. There is no confidence counter that could absorb such noise.
- **Cannot learn alternating or periodic strides.** Patterns whose stride sequence is periodic but not constant, such as $\{+1, +3, +1, +3, \dots\}$, never produce two equal consecutive strides, so no prefetch is ever issued even though the pattern is fully predictable. Typical sources are structure-of-fields traversals, padded 2D arrays and nested loops. Only one delta of history is kept, so such patterns cannot be captured.
- **No support for irregular or indirect accesses.** Pointer chasing and indirect accesses such as `A[B[i]]` have no constant stride, so the prefetcher gives no benefit and only trains on noise.
- **Training delay.** An IP needs three accesses before its first prefetch, which hurts short loops and short streams.
- **Page boundary.** Prefetching stops at the 4 KB page boundary and each new page has to be trained again, so the lookahead is truncated near the end of every page.
- **Fixed degree and no feedback.** The degree is fixed and is not adapted to accuracy or timeliness, so the prefetcher may be too aggressive on irregular code (cache pollution, wasted bandwidth) or too shallow on regular code.
- **Per-IP table pressure.** Training state is kept per IP in a small LRU table. Programs with many load IPs thrash it, and patterns learned by one IP cannot be shared with other IPs that behave identically.

SPP addresses these by keeping a history of several recent deltas (a signature) and by predicting the next delta from that history, so alternating patterns can be captured and a single irregular access does not destroy the learned state.

1.1.3 Results

All runs use 25M warmup and 25M simulated instructions, with LRU at the L2C. The baseline is the same configuration with no L1D prefetcher. Speedup is $\text{IPC}_{\text{IP-stride}}/\text{IPC}_{\text{baseline}}$, and coverage is (L1D load $\text{misses}_{\text{base}}$ − L1D load $\text{misses}_{\text{IP-stride}}$)/L1D load $\text{misses}_{\text{base}}$. Accuracy is the `ACCURACY` value printed by ChampSim for the L1D, which is the number of useful prefetches divided by the number of prefetches *issued to the lower level* (the prefetches that missed in the L1D and went to the L2), not by all prefetches requested by the prefetcher.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1-001	Baseline	0.2323	1.000	10.68	–	–
	IP-stride	0.2333	1.005	10.12	53.0	5.3
Trace2-002	Baseline	0.2506	1.000	13.54	–	–
	IP-stride	0.2505	1.000	13.44	38.2	0.7
Trace3-001	Baseline	0.3820	1.000	3.40	–	–
	IP-stride	0.3829	1.002	3.32	16.0	2.5

Table 1: Baseline versus IP-stride (degree 3) on the three traces.

Table 1 shows that IP-stride is almost neutral on these traces. Table 2 gives the prefetch counters behind the accuracy and coverage numbers.

Trace	Queued	Hit in L1D	To lower level	Useful	Useless	Late
Trace1-001	155 922	102 542	30 956	16 408	8 834	2 807
Trace2-002	78 701	57 796	11 426	4 361	6 300	272
Trace3-001	103 500	70 396	17 007	2 713	10 595	3 067

Table 2: L1D prefetch counters of IP-stride. "Queued" is the number of prefetch requests put into the L1D prefetch queue (the `L1D PQ ACCESS` line). "Hit in L1D" is the number of queued prefetches whose line was already in the L1D. "To lower level" is the denominator ChampSim uses for accuracy. No prefetch crosses a page boundary.

Observations.

- **Performance gain is negligible.** The speedups are 1.005, 1.000 and 1.002, so IP-stride changes IPC by at most 0.5%.
- **Coverage is low**, only 5.3%, 0.7% and 2.5% of the baseline L1D load misses are removed. Trace2 gains almost nothing, and it is also the trace with the highest baseline L1D load MPKI (13.5), so there is plenty of room that the prefetcher does not use.
- **Accuracy varies a lot between traces**: 53.0%, 38.2% and 16.0%. In Trace3 the useless prefetches (10 595) are almost four times the useful ones (2 713), and the late prefetches (3 067) outnumber the useful ones, so many prefetches also arrive after the demand access.
- **Most prefetches never leave the L1D.** Of the prefetches that look up the L1D, 80.2%,

84.4% and 84.1% (Trace1, 2, 3) find the line already present (the `L1D PREFETCH` hit rate), so they fetch nothing. Only 19.9%, 14.5% and 16.4% of the queued prefetches go to the L2 ("To lower level" over "Queued"). The rest hit in the L1D or merge with a request already in the queue (about 8% of the queued prefetches). The prefetcher therefore re-requests the same lines many times: lookups with no benefit.

- **ChampSim's "ISSUED" counts every prefetch twice.** In this build, `pf_issued` is incremented once when the prefetch enters the queue and again when its translation request is sent to the STLB, so `ISSUED` = `PQ ACCESS` + `PQ TO_CACHE` exactly in all three traces (for Trace1, 155 922 + 142 926 = 298 848). We therefore report the number of queued prefetches instead of "ISSUED".
- **Page-boundary rule works.** There are zero cross-page prefetches in all traces, as the code intends.
- **Trace3 has the least to gain**: its baseline L1D load MPKI is already low (3.40), so even a perfect prefetcher could remove only a small number of misses.

These results are the reference point for the SPP evaluation.

1.2 Task 1B: Stride Pattern Prefetcher (SPP)

1.2.1 Design

The SPP prefetcher keeps the per-IP organisation of the IP-stride prefetcher, but every IP entry holds two predictors instead of one: a cheap *uniform* predictor for constant strides and a small *signature table* for strides that follow a repeating pattern. A confidence counter in each predictor is raised on correct predictions and lowered on wrong ones, so a single irregular access no longer destroys what was learned. On top of this, a small prefetch filter measures how many prefetches are useful, and the prefetch degree is throttled from that measurement.

Overview.

Figure 1 shows the structures. Only demand loads (`type == LOAD`) are used for training and prediction. Strides are always measured in cache lines.

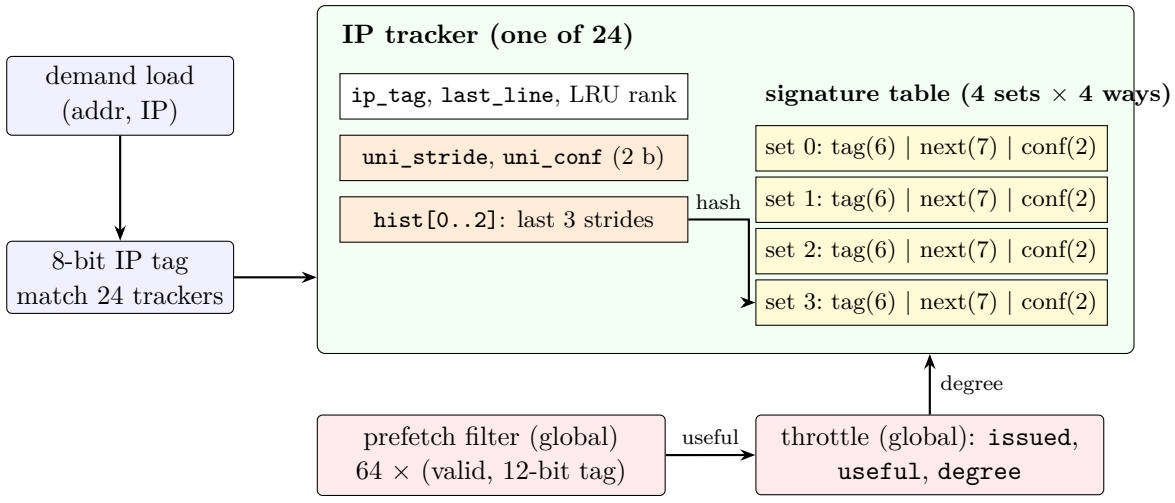


Figure 1: Structures of the SPP prefetcher.

IP tracker.

Like IP-stride, the trackers are a plain array of 24 entries, indexed by an 8-bit IP tag and replaced in LRU order. Besides the IP tag and the last accessed cache line, each tracker holds:

- a **uniform predictor**: a stride `uni_stride` and a 2-bit confidence `uni_conf`;
- a **stride history** `hist[3]` with the last three strides of this IP;
- a private **signature table** of 4 sets $\times$ 4 ways. Each entry holds a 6-bit tag, the stride that followed the signature (`next`, 7 bits signed) and a 2-bit confidence. A confidence of 0 marks an invalid entry.

The *signature* of an IP is the hash of its three-stride history (FNV-style mix of the three strides); two bits of it select the set and six bits form the tag. A signature therefore stands for "the last three strides were a, b, c" and the table learns which stride d usually comes next.

Training.

On every demand load of a known IP the stride $s = \text{line} - \texttt{last_line}$ is computed. A stride of 0 (same line) is ignored. A stride larger than ± 63 lines is treated as a jump: the uniform predictor and the history are cleared and no learning takes place. Otherwise both predictors are trained:

- **Uniform predictor.** If $s =$ `uni_stride`, `uni_conf` is increased (saturating at 3). Otherwise it is decreased by 1 (the miss penalty), and only once it has reached 0 is `uni_stride` replaced by s. A constant-stride stream therefore needs three accesses to reach the usable confidence of 2, but one stray stride only costs one confidence step instead of the whole entry.
- **Signature table.** The signature of the *old* history is looked up. On a hit whose `next` equals s the confidence is increased (saturating at 3), otherwise it is decreased and the entry is cleared when it reaches 0. On a miss the weakest way of the set is chosen: if its confidence is 0 it is replaced by a new entry with $(\text{tag}, s, 1)$, otherwise its confidence is only decreased. Stale or strong entries are thus replaced gradually and not at the first conflict.

Finally s is shifted into `hist` and `last_line` is updated.

Uniform / non-uniform classification and fallback.

An IP is classified as *uniform* when `uni_conf` ≥ 2, i.e. its recent strides have mostly been the same. Otherwise it is *non-uniform* and the signature table is used. This is a fallback chain (Figure 2): constant strides are handled by the cheap uniform predictor, which needs no history, and only IPs that are not uniform are handed to the signature table, which can also follow alternating or otherwise periodic strides. If the signature table has no confident entry for the current history, nothing is prefetched.

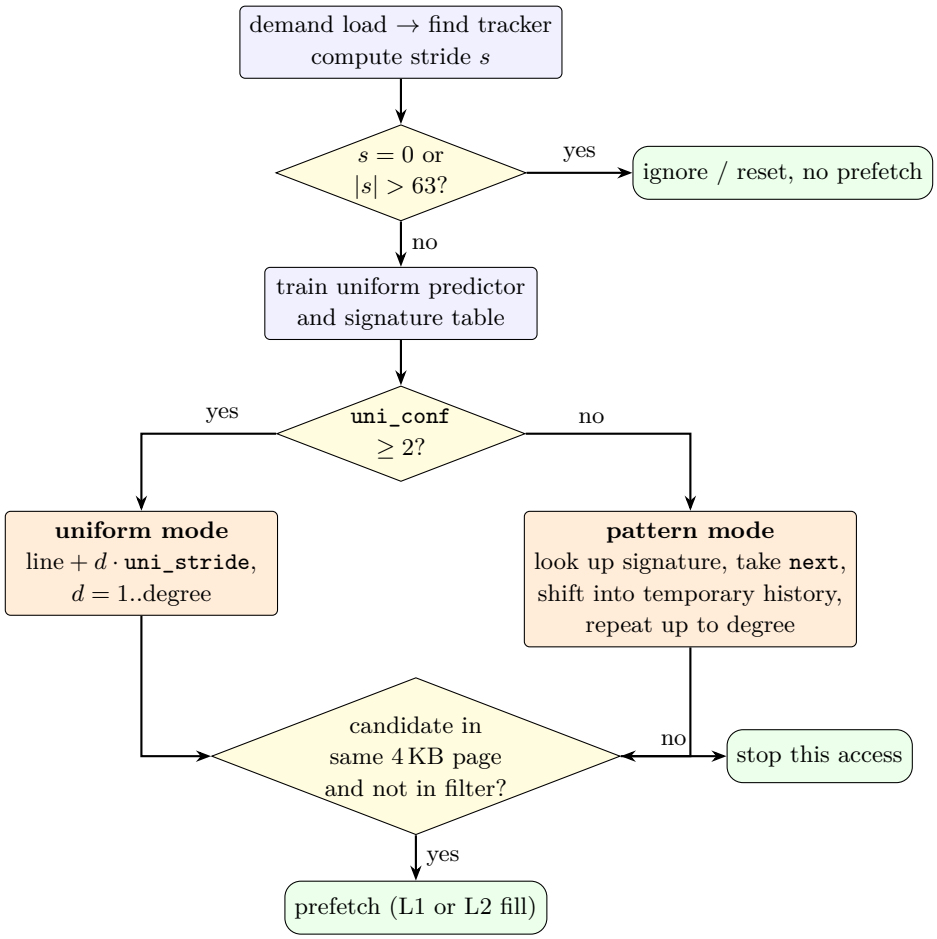


Figure 2: Decision flow of the SPP prefetcher for one demand load.

Prefetch generation, degree and page limit.

In uniform mode up to `degree` lines line + $d \cdot$ `uni_stride` are requested. In pattern mode the predictor walks ahead: it looks up the signature of the current history, takes the predicted `next` stride, adds it to the running line address, shifts it into a temporary copy of the history and repeats, so the lookahead follows the learned pattern step by step and stops as soon as there is no entry of confidence ≥ 1. Every candidate must stay inside the **same 4 KB page** as the demand access, since the next physical page is unknown; the first candidate that crosses the page ends the walk. The fill level depends on the miss-handling pressure: a prefetch fills the L1D when fewer than half of the MSHRs are occupied, and otherwise only the L2C, so prefetches do not starve demand misses.

Prefetch filter and usefulness tracking.

Each issued prefetch is recorded in a 64-entry direct-mapped filter (valid bit and 12-bit tag). It has two roles. First, a candidate already present in the filter is not requested again (duplicate suppression). Second, when a later demand load hits a line recorded in the filter, that prefetch is counted as *useful* and the entry is cleared. This gives the prefetcher its own accuracy, useful/issued, independent of ChampSim's counters.

Accuracy-based throttling.

Issued and useful prefetches are counted over an epoch of 256 issued prefetches. At the end of an epoch the accuracy is computed and the degree for the next epoch is chosen from three levels: a low degree (1) if accuracy is below 40 %, a high degree (4) if it is above 75 %, and a middle degree (2) otherwise; the counters are then reset. Thus the prefetcher is aggressive when its predictions are good and falls back when they are mostly wrong, which limits cache pollution and wasted bandwidth. The choice of the thresholds and degrees is discussed in Section 1.2.3.

Example.

Consider an IP with the stride sequence $+1, +3, +1, +3, \ldots$, which IP-stride can never prefetch. Its uniform predictor keeps being contradicted, so `uni_conf` stays below 2 and the IP is non-uniform. The signature table, however, learns the two entries $(+1, +3, +1) \rightarrow +3$ and $(+3, +1, +3) \rightarrow +1$; after a few repetitions their confidence is ≥ 1 and, from the current history, SPP prefetches $+3, +1, +3, \ldots$ lines ahead, up to the degree. A single irregular stride in a constant stream only lowers `uni_conf` by one, so the stream remains classified as uniform and keeps being prefetched.

1.2.2 Storage overhead

The storage is counted bit-exactly with the logical field widths of the hardware, not the C++ type sizes. Strides are 7-bit signed values (range ± 63) and a line address is $64 - 6 = 58$ bits. Table 3 lists every field.

Table	Field	Bits per entry	Entries	Total bits
Signature entry (inside every tracker)				
	tag	6		
	`next` stride	7		
	confidence	2		
	entry total	**15**	16 per tracker	240 per tracker
IP tracker				
	IP tag	8		
	`last_line`	58		
	`uni_stride`	7		
	`uni_conf`	2		
	`hist[3]` (3×7)	21		
	LRU rank	5		
	signature table (16×15)	240		
	tracker total	**341**	24	**8184**
Prefetch filter				
	valid bit	1		
	tag	12		
	entry total	**13**	64	**832**
Throttle control				
	`epoch_issued` (0–256)	9		
	`epoch_useful` (0–256)	9		
	degree (3 levels)	2		
	total	**20**	1	**20**
Total SPP storage				**9036 bits**
				(1129.5 bytes ≈ 1.10 KB)

Table 3: Storage overhead of SPP.

The trackers dominate the cost: $24 \times 341 = 8184$ bits, i.e. 90.6 % of the total, and about 70 % of each tracker is its private signature table (240 of 341 bits). The filter adds 832 bits and the control state only 20 bits. The statistics counters (`tot_issued` etc.) are only used for the final printout, are not part of the prefetcher logic, and are not counted. The same breakdown is printed by `l1d_prefetcher_final_stats` at the end of each run.

1.2.3 Accuracy-based throttling

Threshold selection.

The throttle works on epochs of $N = 256$ issued prefetches. At the end of an epoch the accuracy is $\text{acc} = 100 \cdot \text{useful/issued}$ (integer percent), computed from the prefetcher's own filter counters (Section 1.2), and the degree for the next epoch is

$$\text{degree} = \begin{cases} 1 & \text{acc} < T_{\text{low}} = 40\,\%, \\ 4 & \text{acc} > T_{\text{high}} = 75\,\%, \\ 2 & \text{otherwise.} \end{cases}$$

The counters are reset after every epoch, so the decision always reflects only the most recent 256 prefetches. The epoch is counted in issued prefetches rather than in cycles or accesses, so the accuracy estimate always has the same sample size, and 256 is large enough that one noisy prefetch moves the estimate by less than 0.4 percentage points.

The two thresholds split the accuracy range into three regions with a different cost of a wrong prefetch. Below 40 % most prefetches are wasted, and each wrong prefetch occupies an MSHR and L2/L1 bandwidth and can evict a useful line, so the prefetcher is throttled to the minimum. Above 75 % at least three of four prefetches are used, so going deeper is worth it. In between the effect of the degree is unclear and a moderate value is kept. The measured accuracy of IP-stride on our traces (53 %, 38 % and 16 %, Table 1) also lies around and below the lower threshold, which shows that a low-accuracy region needs to be handled explicitly.

How the thresholds were decided.

The starting values 40 % and 75 % came from the reasoning above. We then checked them experimentally. All other SPP parameters were kept at their final values, and only the throttle was changed. Each setting was run on all three traces (25 M warm-up + 25 M simulated instructions). Table 4 compares the chosen setting with three other threshold pairs and two other degree sets. Speedup and coverage are relative to the no-prefetching baseline. Accuracy is ChampSim's L1D accuracy, as in Task 1A.

		Speedup				Coverage (%)			Accuracy (%)		
$T_{\text{low}}/T_{\text{high}}$	Degrees	T1	T2	T3	Geomean	T1	T2	T3	T1	T2	T3
40/75	**1/2/4**	**1.0042**	**0.9997**	**1.0038**	**1.0026**	5.4	0.7	4.3	57.0	46.6	26.1
30/60	1/2/4	1.0041	0.9995	1.0039	1.0025	5.6	0.7	4.5	55.9	43.5	24.8
50/85	1/2/4	1.0040	0.9998	1.0038	1.0025	5.1	0.7	3.8	58.2	50.3	27.0
60/90	1/2/4	1.0039	0.9999	1.0034	1.0024	5.0	0.7	3.6	58.6	51.6	27.2
40/75	1/3/6	1.0040	0.9996	1.0039	1.0025	5.8	0.7	4.4	54.9	42.9	23.7
40/75	2/4/8	1.0037	0.9994	1.0038	1.0023	5.9	0.7	4.7	52.5	40.3	22.8

Table 4: Throttle sweep: threshold pairs and degree sets (T1–T3 = Trace1-001, Trace2-002, Trace3-001).

Lower thresholds (30/60) keep the prefetcher at degree 4 more often. Coverage rises slightly, but accuracy drops by 1–3 points and the geometric-mean speedup does not improve. Higher thresholds (50/85, 60/90) move many epochs from degree 4 to degree 2. Accuracy rises, but coverage falls, most clearly on Trace3 (4.3 % → 3.6 %), and the speedup drops. The pair 40/75

has the highest geometric-mean speedup, so we kept it. The differences between all settings are small (below 0.03 % in geometric-mean speedup). The traces offer little headroom for an L1D prefetcher (Section 1.1), so the throttle mainly changes the number of prefetches, not the IPC.

Which accuracy the throttle sees.

The throttle uses the prefetcher's own filter counters, not ChampSim's accuracy. The two are very different (Table 5). The filter counts a prefetch as useful whenever a later demand load touches that line, even if the line was already in the L1D and the prefetch was dropped. So the own accuracy (80–86 %) is much higher than ChampSim's (26–57 %). As a result, the throttle chooses degree 4 in most epochs and almost never degree 1. The thresholds therefore act on the own-accuracy scale. With 40/75, 75 % on this scale separates the regular phases (degree 4) from the noisier ones (degree 2). The lower threshold of 40 % is a safety net for phases where most prefetches are wrong, and such phases are rare in these traces.

Trace	Own accuracy (%)	ChampSim accuracy (%)	Epochs at degree		
			1	2	4
Trace1-001	86.3	57.0	1	154	689
Trace2-002	83.3	46.6	5	62	190
Trace3-001	79.6	26.1	1	198	413

Table 5: Own (filter) accuracy, ChampSim accuracy and degree chosen per epoch for the final SPP (40/75, degrees 1/2/4). Epochs include the warm-up phase.

Choice of prefetch degrees.

The three levels are chosen to give the throttle a clear role for each accuracy region:

- **Degree 1 (accuracy $< 40\,\%$).** The lowest degree that still keeps the prefetcher running. It prefetches only the next predicted line, the one with the highest confidence, so the cost of wrong predictions is minimal, while the prefetcher can still recover when the program enters a regular phase. The prefetcher is not switched off completely, because then no new usefulness would be measured and the degree could never rise again.
- **Degree 2 (between 40 % and 75 %).** The default and the start value. It hides more latency than degree 1 while limiting the waste if some predictions are wrong.
- **Degree 4 (accuracy $> 75\,\%$).** For regular streams, where the lines farther ahead are almost always used. A deeper lookahead is needed to be timely, since the prefetch has to complete before the demand access arrives. Four is the maximum, because it is kept within the page limit (a 4 KB page has only 64 lines) and because a large degree raises the pressure on the MSHRs and the prefetch filter (64 entries).

The degrees are in a $1 : 2 : 4$ ratio so that each step is clearly different from the next.

To check where the useful range of degrees ends, we also ran SPP with throttling disabled, using the same degree in every epoch (Table 6).

	Speedup				Coverage (%)			Accuracy (%)			L1D pf
Degree	T1	T2	T3	Geomean	T1	T2	T3	T1	T2	T3	requests (T1)
1	1.0032	1.0000	1.0023	1.0018	3.7	0.6	2.3	56.2	55.9	28.2	100 003
2	1.0039	0.9998	1.0032	1.0023	4.8	0.7	3.3	59.2	53.1	26.3	123 343
3	1.0041	0.9998	1.0037	1.0025	5.3	0.7	3.9	57.6	48.5	24.8	134 486
4	1.0042	0.9997	1.0039	1.0026	5.7	0.8	4.6	55.6	44.6	24.7	144 342
6	1.0036	0.9995	1.0037	1.0023	6.0	0.8	4.9	51.5	39.9	21.6	160 979
8	1.0032	0.9996	1.0034	1.0021	6.1	0.9	5.0	47.9	38.0	18.7	176 682
throttled 1/2/4	**1.0042**	**0.9997**	**1.0038**	**1.0026**	5.4	0.7	4.3	57.0	46.6	26.1	136 716

Table 6: Fixed prefetch degree (throttling off) compared with the throttled SPP.

The fixed-degree sweep supports the three levels:

- **Speedup peaks at degree 4.** It rises from degree 1 (1.0018) to degree 4 (1.0026) and falls again at 6 and 8. Beyond degree 4 the extra prefetches still add a little coverage (5.7 % → 6.1 % on Trace1), but accuracy keeps falling (55.6 % → 47.9 %) and the wasted prefetches cost more than the new hits gain. Degree 4 is therefore the right maximum.
- **Accuracy falls with every step of degree.** Lines further along the predicted sequence are less certain. This is why a low-accuracy phase should drop to degree 1: it has the highest accuracy on Trace2 and Trace3 and the fewest requests.
- **The throttled 1/2/4 setting matches the best fixed degree with fewer prefetches.** It has the same geometric-mean speedup as fixed degree 4 (1.0026), with 1–2 points higher accuracy on every trace and 5 % fewer L1D prefetch requests on Trace1 (136 716 vs. 144 342). The 1/3/6 and 2/4/8 sets (Table 4) go past degree 4 in good phases and are slightly worse,

which agrees with the fixed-degree results.

1.2.4 Results

All runs use the same setup as Task 1A: 25M warm-up and 25M simulated instructions, LRU at the L2C, and the same definitions of speedup, coverage and accuracy (Section 1.1). SPP uses the final parameters of Section 1.2: 24 trackers with 8-bit IP tags, a history of 3 strides, uniform threshold 2 with miss penalty 1, signature confidence ≥ 1, and throttling with 40/75 and degrees 1/2/4. Table 7 gives the main metrics. The geometric-mean speedup over the three traces is 1.0021 for IP-stride and 1.0026 for SPP.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1-001	Baseline	0.2323	1.0000	10.68	–	–
	IP-stride	0.2333	**1.0045**	10.12	53.0	5.3
	SPP	0.2333	1.0042	**10.10**	**57.0**	**5.4**
Trace2-002	Baseline	0.2506	1.0000	13.54	–	–
	IP-stride	0.2505	0.9995	**13.44**	38.2	**0.7**
	SPP	0.2506	**0.9997**	**13.44**	**46.6**	**0.7**
Trace3-001	Baseline	0.3820	1.0000	3.40	–	–
	IP-stride	0.3829	1.0023	3.32	16.0	2.5
	SPP	0.3835	**1.0038**	**3.26**	**26.1**	**4.3**

Table 7: Task 1 results: baseline (no L1D prefetcher), IP-stride and SPP. The best prefetcher value per trace is in bold.

Trace	Prefetcher	Queued	Hit in L1D	Merged	To lower level	Useful	Late
Trace1-001	IP-stride	155 922	102 542	12 994	30 956	16 408	2 807
	SPP	68 498	37 621	258	29 211	16 649	2 998
Trace2-002	IP-stride	78 701	57 796	5 750	11 426	4 361	272
	SPP	42 488	33 818	71	8 259	3 845	270
Trace3-001	IP-stride	103 500	70 396	6 362	17 007	2 713	3 067
	SPP	52 599	33 272	343	15 807	4 127	2 647

Table 8: L1D prefetch counters of IP-stride and SPP (columns as in Table 2; "Merged" is the number of queued prefetches merged with a request already in the prefetch queue).

Prefetcher	Trace1-001	Trace2-002	Trace3-001
Baseline	17.75 / 7.35	28.23 / 8.70	3.96 / 2.84
IP-stride	17.22 / 6.83	28.32 / 8.61	3.77 / 2.63
SPP	17.19 / 6.81	28.30 / 8.62	3.71 / 2.57

Table 9: L2C load MPKI / L2C data load MPKI. The L1D prefetchers also reduce data misses at the L2C, because prefetches that are not timely still bring the line closer.

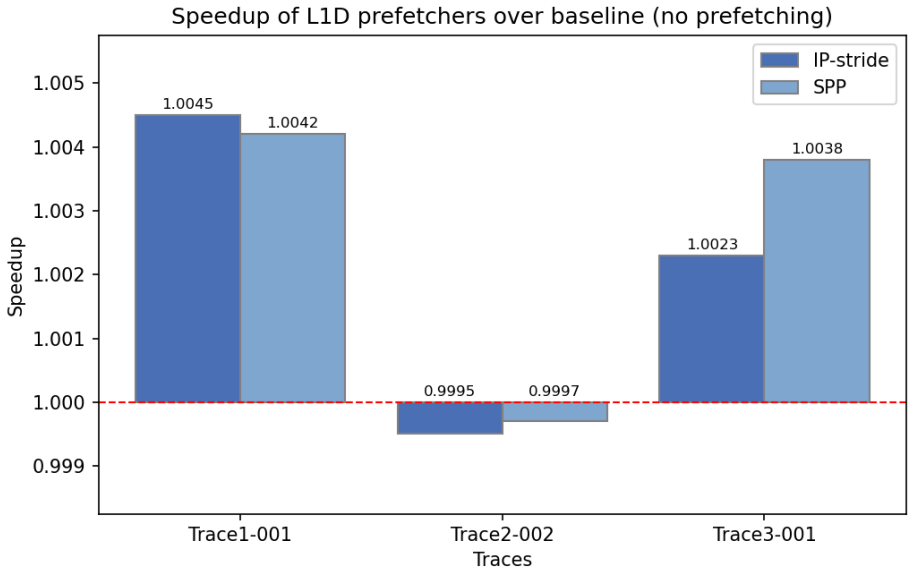


Figure 3: Speedup over the no-prefetching baseline (axis zoomed around 1.0).

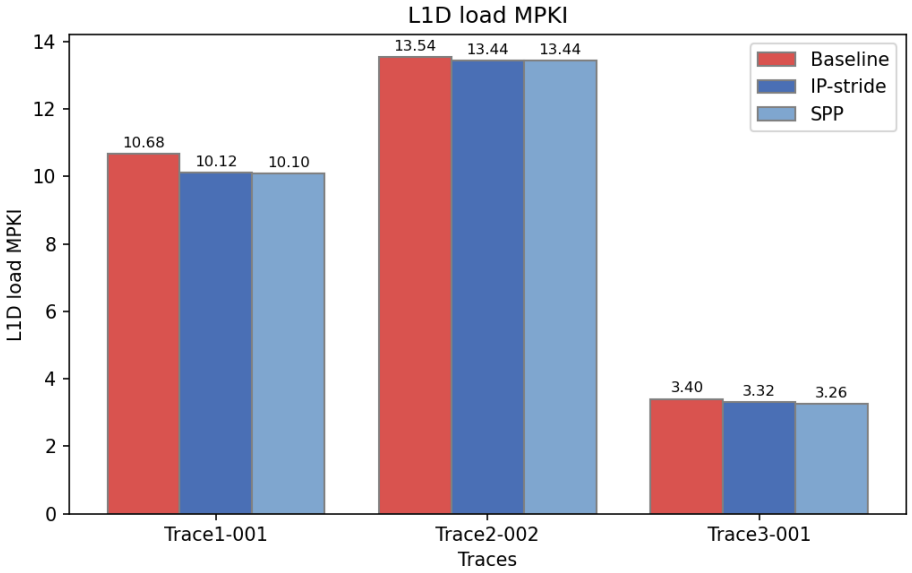


Figure 4: L1D load MPKI.

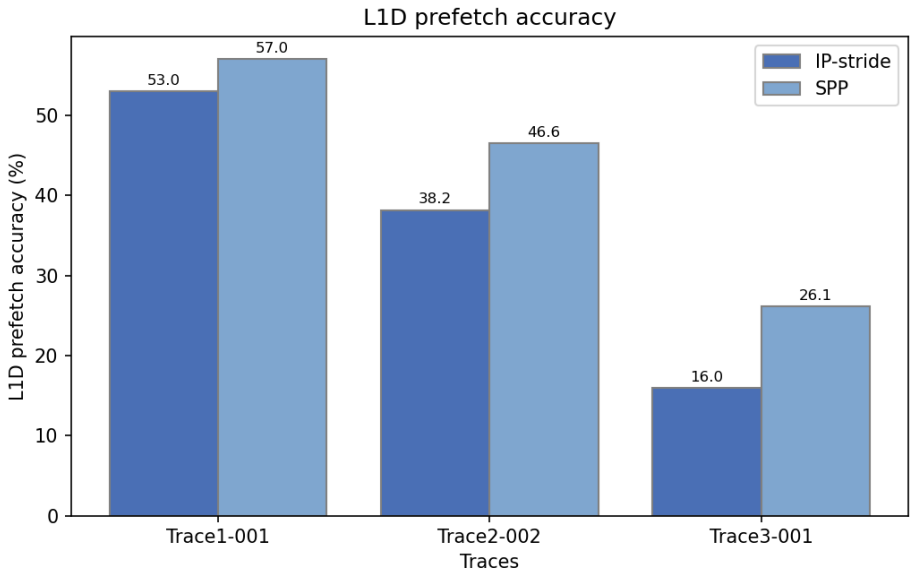


Figure 5: L1D prefetch accuracy (ChampSim definition).

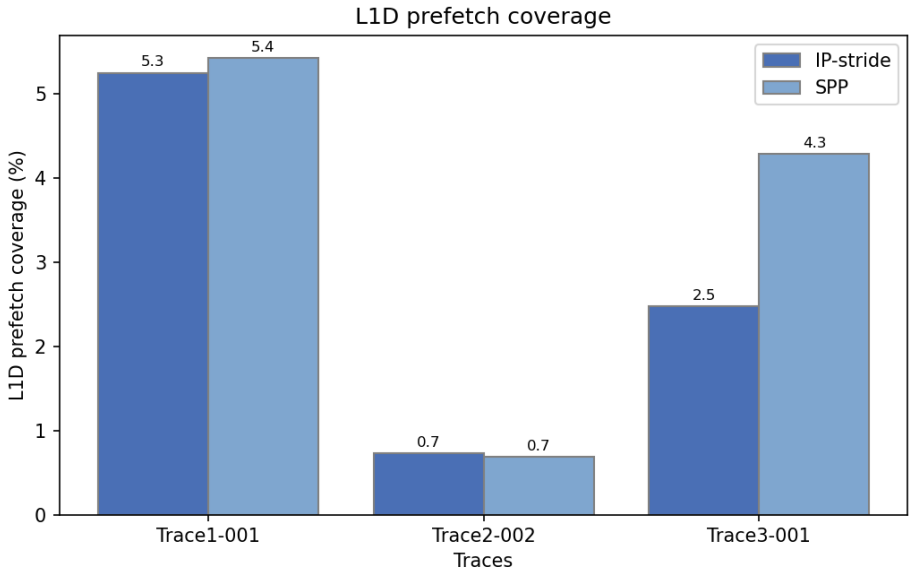


Figure 6: L1D prefetch coverage (reduction of L1D load misses relative to the baseline).

Observations.

- **SPP is more accurate on every trace**: 57.0%, 46.6% and 26.1% against 53.0%, 38.2% and 16.0% for IP-stride.
- **SPP reaches the same or higher coverage with about half the prefetches.** It queues 56%, 46% and 49% fewer prefetches than IP-stride, but its coverage is the same on Trace1 and Trace2 and much higher on Trace3 (4.3% against 2.5%). On Trace3 it removes 3 648 L1D load misses against 2 110 for IP-stride, and it has 52% more useful prefetches (4 127 against 2 713).
- **Far fewer redundant prefetches.** Only 258, 71 and 343 prefetches are merged in the queue,

against 12 994, 5 750 and 6 362 for IP-stride, because the prefetch filter drops candidates that were requested recently. The share of queued prefetches that find the line already in the L1D also falls (from 80% to 61% on Trace1).

- **Speedup is still small.** The best case is 1.0042 on Trace1 and 1.0038 on Trace3. On Trace1, IP-stride is ahead by 0.0003 even though SPP has slightly higher coverage and accuracy. SPP has a few more late prefetches there (2 998 against 2 807), but a difference this small should not be read as a real advantage of either prefetcher.
- **Trace2 does not benefit from either prefetcher**: both change IPC by less than 0.05% and remove under 1% of its L1D load misses.

1.2.5 Discussion

Where SPP wins against IP-stride, and why.

- **Irregular but repeating strides (Trace3).** IP-stride only prefetches when the same stride appears twice in a row. SPP's signature table also learns strides that follow a repeating pattern. On Trace3 39% of SPP's prefetches come from the pattern path, and this is the trace where SPP gains most: coverage 2.5% → 4.3%, accuracy 16.0% → 26.1% and speedup 1.0023 → 1.0038.
- **Tolerance to single wrong strides.** IP-stride replaces its stride after every mismatch, so one irregular access stops its prefetching until the stride repeats again. SPP's confidence counter only loses one step, so a stream with occasional noise keeps being prefetched.
- **Throttling and filtering.** The filter and the degree throttle cut the number of queued prefetches roughly in half without losing coverage. This saves L1D tag lookups, prefetch-queue slots and L2 bandwidth, which IP-stride (fixed degree 3) spends on lines that are already present.

Where SPP does not win, and why.

- **Constant-stride streams (Trace1).** For streams that IP-stride already handles, SPP can only match it. Both reach about 5.4% coverage, and the speedup difference (0.0003) is within the noise of these runs.
- **Misses that are not stride-predictable (Trace2).** No variant we tried moved Trace2 above 0.7% coverage. Its misses come from IPs whose consecutive addresses have no stable stride or short stride pattern, which a per-IP stride prefetcher cannot predict.

Why the speedups are small for both prefetchers.

- **The traces are front-end bound.** The baseline L1I MPKI is 14.5, 40.0 and 2.1, and the branch MPKI is 16.1, 6.3 and 7.2. Trace1 and Trace2 spend much of their time waiting for instructions or recovering from mispredictions, so removing a few percent of the data misses changes the IPC very little. Trace3 has few misses of any kind (L1D load MPKI 3.4), so there is little left to remove.
- **Only a small part of the misses is stride-predictable.** An offline analysis of the load streams showed that only about 6–12% of the loads can be predicted by a per-IP uniform or signature scheme even with unlimited tables, which bounds the coverage of both prefetchers.

- **A small tracker table.** Each trace has 73k–170k distinct load IPs, and 24 trackers hold the IP of only about one load in five. An earlier experiment with the initial SPP parameters showed that a larger table helps: 128 trackers gave speedups of 1.004/1.002/1.005, and 512 trackers 1.006/1.007/1.015. But 512 trackers need about 23 KB, against 1.1 KB for the final design. We kept 24 trackers to stay lightweight and to compare with IP-stride at the same number of entries.

How the parameters were tuned.

The first version of SPP used a 5-stride history, a uniform threshold of 3 with miss penalty 2, signature confidence ≥ 2 and 12-bit IP tags. It was more conservative than IP-stride and had *lower* coverage. Table 10 shows how each change moved the results. The main reason for the low initial coverage was the long training time: with short strided runs, SPP was often still learning when the run ended. A shorter history and lower thresholds fix this at a small cost in accuracy, and 8-bit IP tags slightly increase coverage while saving 96 bits.

	Speedup			Coverage (%)			Accuracy (%)			Storage
Version	T1	T2	T3	T1	T2	T3	T1	T2	T3	(bits)
Initial (history 5, thr. 3, pen. 2, sig. ≥ 2)	1.003	1.000	1.002	3.6	0.6	2.3	59.2	60.9	33.4	9 468
+ history 3, thr. 2, pen. 1	1.004	1.000	1.002	4.9	0.6	2.5	58.6	53.3	21.9	9 132
+ signature confidence ≥ 1	1.004	1.000	1.004	5.3	0.7	4.2	56.9	48.8	25.9	9 132
+ 8-bit IP tag (final)	1.004	1.000	1.004	5.4	0.7	4.3	57.0	46.6	26.1	9 036

Table 10: Effect of the parameter changes; each row adds one change to the previous row.

2 Task 2: Cache Replacement Policy

2.1 Task 2A: Instruction-conscious L2 replacement (I-RRIP)

2.1.1 Design

Passing the instruction bit.

`update_replacement_state` now takes an extra last argument `uint8_t isInstruction`. The declaration in `inc/cache.h`, every `replacement/*` file and every call site in `src/cache.cc` and `src/ooo_cpu.cc` were updated. Call sites pass the `instruction` field of the packet being filled or hit.
The policy treats a request as an instruction request only if `isInstruction && (type == LOAD || type == PREFETCH)`. This is the same rule `fill_cache` uses to set `block.instruction` in the L2C.
`find_victim` does not receive the flag, so it reads `current_set[way].instruction`.

Policy

(`replacement/irrip.l2c_repl`). I-RRIP is SRRIP with a 2-bit RRPV (0 = near, 3 = distant) that treats instruction lines, data lines and writebacks differently.

Event	SRRIP (given `rrip.l2c_repl`)	I-RRIP
Fill, instruction line	2	2
Fill, data line	2	3
Fill, writeback	2	3
Hit, instruction line	0	0
Hit, data line	0	2
Hit, writeback	0	unchanged
Victim	first line at 3	first *data* line at 3, else first line at 3

Table 11: RRPV assigned on each event. If no line is at 3, all RRPVs in the set are incremented and the search repeats.

Storage.

2 bits × 1024 sets × 8 ways = 2 KB, the same as SRRIP. The instruction bit is already stored with each L2C block, so no new state is needed.

How the values were chosen.

The six values above and the eviction preference were parameters that we changed from run to run to observe how the speedup and MPKI change. We ran simulations on combinations with the same philosophy: that we should prioritize instruction lines staying in the cache over data lines. This means that we always inserted instruction lines with an RRPV value that was $\leq$ the insertion RRPV of data. We then checked the average L2 instruction MPKI and speedup and

chose the best one accordingly.

2.1.2 Rationale

The table below starts from the final policy and changes one decision at a time (geomean speedup over LRU; MPKIs are averages over the three traces). The last rows are the starting point we tried first and RRIP-like settings.

Policy	Speedup	L2 instr. MPKI	L2 data MPKI
Final I-RRIP	1.0216	5.94	8.02
Eviction: plain SRRIP victim (no data-first)	1.0214	6.13	7.87
Promote data hit to 3 (no protection)	1.0203	5.94	8.55
Promote data hit to 1 instead of 2	1.0203	6.23	7.77
Promote data hit to 0 instead of 2	1.0191	6.51	7.59
Promote instruction hit to 1 instead of 0	1.0174	6.16	8.00
Insert writeback at 2 instead of 3	1.0157	6.37	7.56
Insert instruction at 1 instead of 2	1.0148	6.33	8.07
Insert instruction at 0 instead of 2	1.0118	6.59	8.10
Insert data at 2 instead of 3	1.0114	7.32	6.75
First attempt (instr 0, data 2, WB 3, promote 0/0)	1.0076	8.09	6.75
All fills at 2, no priority, no WB promotion	1.0045	9.33	6.04
Given SRRIP (`rrip.l2c_repl`)	1.0008	10.19	6.01

Table 12: One-at-a-time changes from the final policy.

Reading the RRPV.

The RRPV of a line is a prediction of how soon it will be reused: 0 is kept longest, 3 is the next victim. A fill enters at an intermediate value and has to be hit to be promoted. When no line in the set is at 3, every RRPV in the set is incremented by one (an *aging step*) and the search repeats. This happens only on a miss. So the RRPV a class of lines gets on fill and on hit decides how many aging steps (misses to that set) it survives. Table 13 lists this for I-RRIP.

Line class	RRPV	Aging steps before it can be evicted
Writeback fill, or data fill that is never hit	3	0
Instruction fill, never hit	2	1
Data line after a hit	2	1
Instruction line after a hit	0	3

Table 13: Protection given to each class of line in I-RRIP. Within the same RRPV, data lines are evicted before instruction lines. A data line never ranks above an instruction line.

Insertion (modified, the largest effect).

SRRIP inserts every line at 2. The idea behind inserting below the maximum protection is scan resistance: a line that was just fetched has shown no reuse yet, so it should not displace lines that have. Two properties of the L2 make the type of a line a useful extra signal, and I-RRIP uses them to choose a different insertion value for each type.

- **Instruction lines reuse more, and their misses cost more.** The code footprint is larger than the L1I, so code that the L1I evicts is fetched again, and the L2 is where it is found. Code is also executed repeatedly (loops, shared library functions). An instruction miss stalls fetch and decode, so the core has nothing to execute, while an out-of-order core can overlap part of a data miss with other work. Both facts argue for keeping instruction lines longer.
- **Data lines at the L2 are often not reused.** The L1D has already absorbed the short-range temporal reuse, so a data line filled into the L2 is often touched once (streaming or scan-like accesses). Keeping such a line costs a way that an instruction line could use.
- **Writebacks are not demand requests.** A written-back line only moves the L1D's dirty copy down; nothing shows that the program will read it from the L2 again soon.

This gives the insertion values: *data and writebacks at 3* (next victim, they leave first unless a hit proves reuse) and *instruction lines at 2* (one aging step of grace). The data value is the most important decision in the policy. Placing data at 2 instead keeps one-shot data around for an extra aging step, so it competes with instructions and the speedup falls from 1.0216 to 1.0114 and the average instruction MPKI rises from 5.94 to 7.32 (Table 12). The cost of data at 3 is that some data lines which would have been reused after a longer gap are lost, so the data MPKI rises. That is the intended trade, because the instruction misses saved are more costly. Writebacks behave the same way: at 2 instead of 3 they push the instruction MPKI up (5.94 to 6.37) and the speedup down (1.0157).

Instruction lines are deliberately *not* inserted at 0, even though 0 looks like stronger priority. The RRPV only carries information if a fresh line and a line that has been hit have different values. At 0 they look the same, and equal RRPVs are resolved by way index (the victim search takes the first match), not by recency. A code line that is never fetched again then stays as long as hot code. Inserting at 0 gives a speedup of 1.0118 and at 1 of 1.0148, against 1.0216 at 2, and Trace1 falls below LRU (0.994 at 0). On Trace2 the instruction MPKI is higher at 0 than at 2 (9.80 against 8.22), so the extra priority does not even reduce instruction misses. At 2 a line has to be hit to reach 0, so the RRPV separates proven code from unproven code.

Promotion (modified).

A hit is the only evidence of reuse that the policy gets, and SRRIP uses it by resetting the RRPV to 0 (hit priority). I-RRIP keeps that for instruction lines and weakens it for data. The strength of the promotion follows how reliable the evidence is and what a miss on the line would cost.

- **Instruction hit → 0.** A code line that is hit again is likely to be hit again, and its miss is expensive, so it gets the full promotion to RRPV=0. Promoting to 1 instead removes one step; the speedup falls to 1.0174 and the instruction MPKI rises to 6.16.
- **Data hit → 2.** A single hit on a data line is weaker evidence than a hit on code, and the line is cheaper to lose, so we tried all four values. Promoting to 0 or 1 protects a once-hit data line for three or two aging steps. That is as long as, or longer than, a fresh instruction line, so data crowds out code (instruction MPKI 6.51 and 6.23, speedup 1.0191 and 1.0203). At 2 a hit keeps the line for one aging step, like a fresh instruction line, and data-first eviction settles ties. The instruction MPKI falls to 5.94 and the speedup is the best, 1.0216. At 3 a hit gives no protection. The instruction MPKI stays at 5.94, but the data MPKI jumps to 8.55 and the total L2C load MPKI to 14.48 (13.96 at 2), and the speedup drops to 1.0203. So 2 is the point where a hit still counts but a data line never ranks above an instruction line.
- **Writeback hit → unchanged.** A writeback that finds its line in the L2 is the L1D returning a dirty copy. The program has not read the line from the L2, so promoting it would measure L1D traffic and not L2 reuse. The given SRRIP promotes it to 0, which keeps lines that are not being reused. Leaving the RRPV alone is the only difference between SRRIP and the policy with all fills at 2, and it is worth +0.4% by itself (1.0045 against 1.0008).

Eviction (modified, small effect).

SRRIP evicts the first way at the maximum RRPV, and ages the set when none is there. I-RRIP keeps this order and adds a tie-break: among the ways at RRPV 3, a data line is evicted before an instruction line. The RRPV is the reuse prediction, and when two lines have the same prediction the cheaper miss should go first. The type is therefore a second criterion and not a hard partition. We do not evict data lines first regardless of RRPV, because a stale instruction line (one that was hit long ago and has aged to 3) must be able to leave. Instruction lines are not pinned: the aging step always brings them to 3, and when no data line is at 3 an instruction line is evicted. Invalid ways start at 3 and are not instruction lines, so they are used first.

The tie-break matters because data and instruction lines often share an RRPV: a once-hit data line and a fresh instruction line are both at 2, and a stale instruction line and a one-shot data line are both at 3. Data-first eviction makes the data line leave first in both cases. Without it the speedup is 1.0214 against 1.0216 and the instruction MPKI is 6.13 against 5.94 (Trace2: 8.68 against 8.22). The effect is small and not uniform: Trace1 is marginally better without it (1.0062 against 1.0055, with a lower data MPKI of 9.51 against 9.71), but Trace2, Trace3 and the average favour it.

Why this design.

The three changes work together. Insertion and promotion set the ranking, and the eviction preference only breaks ties at RRPV 3. Moving from RRIP to the final policy is worth +2.1% of speedup and a 42% drop in average L2 instruction MPKI (10.19 → 5.94). Settings that give lower data MPKI (e.g. data inserted at 2) lose speedup and raise instruction MPKI. The final policy also has essentially the lowest average L2 instruction MPKI of all 112 policies we simulated (a variant that does not promote data hits is lower by 0.001, 5.936, but has a lower speedup), so both goals (speedup and L2 instruction MPKI) are met by the same policy.

Wider RRPV (not adopted).

We also tried a maximum RRPV of 4, 5 and 7 with the same structure (instruction at max−1, data and writebacks at max). The best 3-bit policy reached 1.0256 (+0.4% over the final 2-bit policy) but needs 3 KB, so we keep the 2-bit design. Run logs (ChampSim output) are in `results/task2/maxrrpv`.

2.1.3 Results

All runs use 25M warmup and 25M simulated instructions. Speedup is $\text{IPC}_{\text{policy}}/\text{IPC}_{\text{LRU}}$ (no L1D prefetcher in all three runs). MPKIs are L2C *load* misses per kilo-instruction, split into instruction and data loads. The "avg" row is the geometric mean for IPC and speedup and the arithmetic mean for MPKI.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2321	1.000	17.73	10.37	7.36
	RRIP	0.2321	1.000	17.66	10.58	7.07
	I-RRIP	0.2334	1.006	18.63	8.92	9.71
Trace2	LRU	0.2500	1.000	28.55	19.84	8.71
	RRIP	0.2508	1.003	26.89	18.78	8.11
	I-RRIP	0.2632	1.053	19.40	8.22	11.17
Trace3	LRU	0.3819	1.000	3.96	1.13	2.83
	RRIP	0.3816	0.999	4.04	1.20	2.84
	I-RRIP	0.3845	1.007	3.85	0.67	3.18
avg	LRU	0.2809	1.000	16.75	10.45	6.30
	RRIP	0.2811	1.001	16.20	10.19	6.01
	I-RRIP	0.2869	1.022	13.96	5.94	8.02

Table 14: Task 2A results.

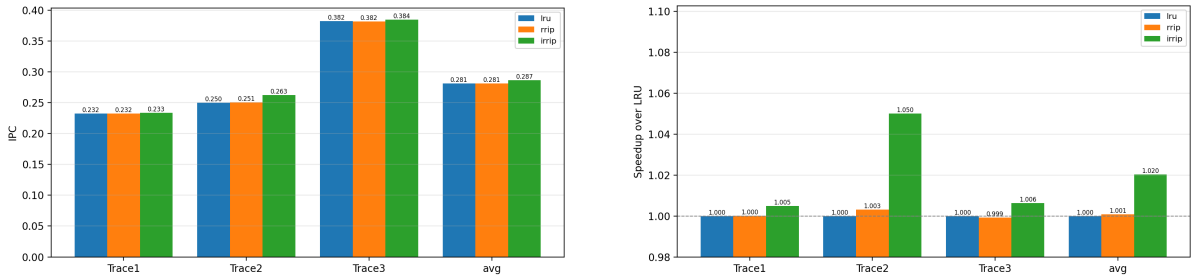


Figure 7: IPC (left) and speedup over LRU (right).

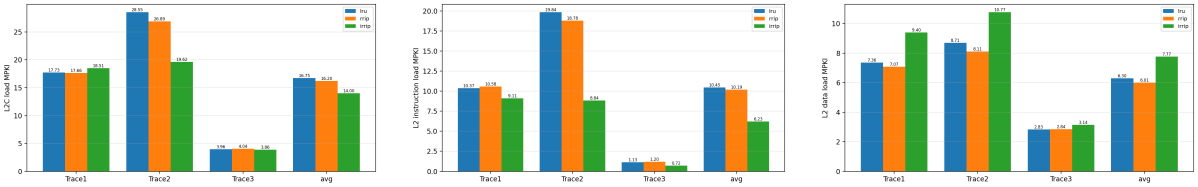


Figure 8: L2C load MPKI: total (left), instruction (middle), data (right).

Observations.

- **Instruction MPKI drops on every trace**: by 14% (Trace1), 59% (Trace2) and 41% (Trace3) compared to LRU, and 42% on average compared to RRIP.
- **Data MPKI rises on every trace** (+32%, +28%, +12% over LRU), which is the cost of the priority.
- **Trace2 gets almost all the speedup** (1.053): its instruction MPKI is the highest (19.8), so it has the most to gain, and the total L2C load MPKI falls from 28.6 to 19.4.
- **Trace1 improves IPC although total L2C load MPKI goes up** (17.73 → 18.63). Its instruction misses fall by 1.45 MPKI but data misses rise by 2.35 MPKI. It might be because an instruction miss stalls the front end, while out-of-order execution can hide part of a data-miss latency.
- **Trace3 changes little** (1.007): its instruction MPKI is only 1.1 under LRU, so there are few instruction misses to remove.
- **RRIP alone gives no gain** (1.001); the benefit comes from the instruction-aware changes.

2.2 Task 2B: Dynamic I-RRIP

2.2.1 Design

[TODO: set dueling, leader sets, decision formula]

2.2.2 Rationale for the leader sets and decision logic

[TODO: number of leader sets and why; decision logic and why]

2.2.3 Sensitivity study

[TODO: 5 values of the decision parameter (e.g. threshold) and their effect]

Parameter value	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
[TODO:]					

Table 15: Sensitivity of Dynamic I-RRIP to its decision parameter.

2.2.4 Results

[TODO: IPC, speedup, Load MPKI of L2C, L2 Instruction and L2 Data]

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
[TODO:]						

Table 16: Task 2B results.

[TODO: plots]

3 Task 3: Incorporating SPP with Dynamic I-RRIP

3.1 Design

[TODO: prefetch awareness in `replacement/dyn_irrip.l2c_repl`]

3.2 Rationale for handling prefetch packets

[TODO: why the replacement policy treats prefetch packets this way]

3.3 Results

[TODO: IPC, speedup, prefetch accuracy, coverage, Load MPKI of L1D, L2C, L2 Instruction and L2 Data]

Trace	Config	IPC	Speedup	Acc. (%)	Cov. (%)	L1D MPKI	L2C MPKI	L2 instr. MPKI	L
[TODO:]									

Table 17: Task 3 results: SPP + Dynamic I-RRIP versus SPP alone and Dynamic I-RRIP alone.

[TODO: plots]

3.4 Discussion

[TODO: comparison against standalone SPP and standalone Dynamic I-RRIP]